Table des matières

Labo 09 OLAP — Séries temporelles et as-of joins sur l'entrepôt 1
Le terrain, en une minute 1
Partie 1 — Le grain temporel, avant tout le reste 2
Partie 2 — Le contrôle que personne ne fait 4
Partie 3 — L'as-of join 5
Partie 4 — Séries, axe complet et cadres 8
Partie 5 — Quantiles, sessions et cohortes 12
Partie 6 — Ce que l'entrepôt vous a montré 14
Nettoyage 15
Et ensuite ? 15

Labo 09 OLAP — Séries temporelles et as-of joins sur l'entrepôt

Laboratoire du Cours 09, voie OLAP · DuckDB + AdventureWorks DW · Formation SQL avancé

Ce laboratoire pose les questions du temps sur l'entrepôt : la série quotidienne, la moyenne mobile, la valeur en vigueur au moment de la vente, les sessions et les cohortes. Comptez une heure et demie.

Les deux voies — Ce labo a un jumeau, `Labo_09_OLTP_SQL.md`, qui pose les mêmes questions sur PostgreSQL. Cette fois la différence n'est pas seulement syntaxique : l'historique du produit vit ici replié dans la dimension, et ses dates de validité sont cassées. Vous allez donc commencer par un contrôle de qualité, ce qui est l'ordre normal des choses même quand on ne s'y attend pas.

Ce que ce labo démontre chiffres en main — L'écriture par intervalle fermé renvoie exactement zéro ligne sur cette dimension, sans lever d'erreur. L'as-of join correct, lui, fonctionne, et révèle au passage que 6 899 lignes de fait sur 60 398 pointent vers une version du produit qui n'est pas celle que les dates de la dimension désignent.

Le terrain, en une minute

```
USE dbo;
```


© 2026 iAlexMG. Tous droits réservés.

Table	Ce qu'elle contient	Colonnes utiles
`dbo.FactInternetSales`	60 398 lignes de vente en ligne	`ProductKey`, `OrderDateKey`, `OrderDate`, `CustomerKey`, `SalesAmount`, `SalesOrderNumber`
`dbo.DimDate`	l'axe des dates, 3 652 jours de 2005 à 2014	`DateKey`, `FullDateAlternateKey`, `CalendarYear`, `MonthNumberOfYear`
`dbo.DimProduct`	le produit, 606 lignes pour 504 produits	`ProductKey`, `ProductAlternateKey`, `ListPrice`, `StandardCost`, `StartDate`, `EndDate`, `Status`
`dbo.DimCustomer`	le client	`CustomerKey`, `DateFirstPurchase`

Deux colonnes commandent tout ce labo. `ProductKey` est la clé technique de la dimension, celle que le fait porte : elle désigne une version précise du produit. `ProductAlternateKey` est la clé métier, le code du catalogue, celle qui reste la même à travers les versions. Une dimension à évolution lente se navigue toujours par la seconde.

On créera des objets de travail préfixés `LABO09_`, faciles à supprimer à la fin.

Partie 1 — Le grain temporel, avant tout le reste

1.1 Les bornes réelles

Théorie : `Theorie_09_SQL.md`, section 1.

```
SELECT MIN(OrderDate) AS premiere, MAX(OrderDate) AS derniere, COUNT(*) AS lignes
FROM   dbo.FactInternetSales;
```

Attendu : du 29 décembre 2010 au 28 janvier 2014, 60 398 lignes. Notez que la première et la dernière période sont tronquées : décembre 2010 ne compte que trois jours et janvier 2014 s'arrête au 28. Tout graphique mensuel de cette table commence et finit par une falaise qui n'est pas un fait économique.

1.2 Deux clés de date pour la même chose

Théorie : `Theorie_09_SQL.md`, section 1.

```
SELECT OrderDateKey, OrderDate FROM dbo.FactInternetSales LIMIT 5;
```

`OrderDateKey` vaut `20101229` et `OrderDate` vaut `2010-12-29`. L'entier est une clé de jointure vers `dbo.DimDate`, pas une date : lui appliquer une fonction de date échouerait, et lui soustraire 1 ne donnerait pas la veille au tournant du mois. C'est la première confusion de grain du module, et elle est ici tendue comme un piège.

© 2026 iAlexMG. Tous droits réservés.

1.3 Ce que dit la dimension produit

Théorie : `Theorie_09_SQL.md`, section 1.

```sql
SELECT COUNT(*)                                  AS lignes,
       COUNT(DISTINCT ProductAlternateKey)       AS produits,
       COUNT(EndDate)                            AS avec_date_de_fin,
       COUNT(Status)                             AS avec_statut
FROM   dbo.DimProduct;
```

Attendu : 606 lignes, 504 produits, 200 dates de fin, 406 statuts. Autrement dit, une ligne porte une date de fin ou un statut, jamais les deux. C'est la signature d'une dimension à évolution lente : les versions périmées portent leur fin, la version courante porte `Current`.

```sql
SELECT nb_versions, COUNT(*) AS produits
FROM   (SELECT ProductAlternateKey, COUNT(*) AS nb_versions
        FROM   dbo.DimProduct GROUP BY ProductAlternateKey)
GROUP  BY nb_versions ORDER BY nb_versions;
```

Attendu : 427 produits à une version, 52 à deux, 25 à trois. Ce sont ces 77 produits qui ont une histoire, et c'est sur eux que tout ce labo se joue.

1.4 Les journées qui ne durent pas 24 heures

Théorie : `Theorie_09_SQL.md`, section 1.

Les dates de l'entrepôt sont des minuits sans fuseau, et ce piège-là ne les touche pas. Il touche toute série horaire d'un lieu qui change d'heure. Comptez les heures du 1er novembre 2026 à Montréal :

```sql
SELECT COUNT(*) AS heures_du_1er_novembre
FROM   generate_series(TIMESTAMP '2026-11-01 00:00:00' AT TIME ZONE 'America/Toronto',
                       TIMESTAMP '2026-11-02 00:00:00' AT TIME ZONE 'America/Toronto' - INTERVAL '1 hour',
                       INTERVAL '1 hour') AS g(h);
```

Attendu : 25. Remplacez les deux dates par le 8 et le 9 mars : 23. Mesurez maintenant la durée de la journée de novembre, de deux façons :

```sql
SELECT (TIMESTAMP '2026-11-02 00:00:00' AT TIME ZONE 'America/Toronto')
     - (TIMESTAMP '2026-11-01 00:00:00' AT TIME ZONE 'America/Toronto') AS soustraction,
       DATE_DIFF('hour',
                 TIMESTAMP '2026-11-01 00:00:00' AT TIME ZONE 'America/Toronto',
                 TIMESTAMP '2026-11-02 00:00:00' AT TIME ZONE 'America/Toronto') AS heures;
```

Attendu : `1 day` et 25. La soustraction compte en jours de calendrier et cache l'heure de plus ; `DATE_DIFF('hour', …)` compte les heures écoulées. Le labo jumeau montre que PostgreSQL, lui, rend `1 day 01:00:00` à la soustraction : la même écriture ne mesure pas la même chose d'un terrain à l'autre.

© 2026 iAlexMG. Tous droits réservés.

Partie 2 — Le contrôle que personne ne fait

C'est la partie qui n'existe pas dans le labo jumeau, parce que la table transactionnelle est saine. Ici, elle change tout.

2.1 Les trois requêtes de contrôle

Théorie : `Theorie_09_SQL.md`, section 1.

```sql
-- 1. Des fins antérieures à leur début ?
SELECT COUNT(*) AS dates_incoherentes
FROM   dbo.DimProduct
WHERE  EndDate IS NOT NULL AND EndDate < StartDate;

-- 2. Des intervalles qui se chevauchent pour un même produit ?
SELECT COUNT(*) AS chevauchements
FROM   dbo.DimProduct AS a
JOIN   dbo.DimProduct AS b
       ON b.ProductAlternateKey = a.ProductAlternateKey
      AND b.StartDate > a.StartDate
      AND b.StartDate <= COALESCE(a.EndDate, '9999-12-31');

-- 3. Plusieurs versions courantes pour un même produit ?
SELECT ProductAlternateKey, COUNT(*) AS courantes
FROM   dbo.DimProduct WHERE EndDate IS NULL
GROUP  BY ProductAlternateKey HAVING COUNT(*) > 1;
```

Attendu pour la première : 200. C'est-à-dire toutes les dates de fin de la table, sans exception. Regardez le cas d'école :

```sql
SELECT ProductKey, ProductAlternateKey, StartDate, EndDate, Status
FROM   dbo.DimProduct
WHERE  ProductAlternateKey = 'HL-U509-R'
ORDER  BY StartDate;
```

ProductKey	StartDate	EndDate	Status
212	2011-07-01	2007-12-28	(vide)
213	2012-07-01	2008-12-27	(vide)
214	2013-07-01	(vide)	Current

La fin précède le début de presque quatre ans, deux fois. Ce n'est pas une donnée exotique choisie pour l'exercice, c'est le contenu réel de la base publiée par l'éditeur.

© 2026 iAlexMG. Tous droits réservés.

2.2 Ce que ça coûte à l'écriture naïve

Théorie : `Theorie_09_SQL.md`, section 1.

```
SELECT COUNT(*) AS lignes_appariees
FROM   dbo.FactInternetSales AS f
JOIN   dbo.DimProduct AS ref ON ref.ProductKey = f.ProductKey
JOIN   dbo.DimProduct AS v
       ON v.ProductAlternateKey = ref.ProductAlternateKey
      AND f.OrderDate BETWEEN v.StartDate AND v.EndDate;
```

Attendu : zéro. Pas une erreur, pas un avertissement, pas une ligne. Une version périmée ne peut apparier personne puisque son intervalle est vide, et la version courante ne le peut pas non plus puisque sa fin est `NULL`. Le rapport bâti sur cette jointure serait donc vide, et si la jointure est un `LEFT JOIN` enfoui dans une couche intermédiaire, il serait plein de `NULL` sans que personne remonte à la cause.

Vous reconnaissez le mode de défaillance : c'est celui du `NOT IN` du Cours 07 et de la valeur `JSON` comparée sans la flèche double du Cours 08. La faute qui coûte cher ne lève pas d'erreur.

Partie 3 — L'as-of join

3.1 L'écriture portable

Théorie : `Theorie_09_SQL.md`, section 6.

Exactement celle du labo jumeau. Elle ne lit jamais `EndDate`, donc le défaut de la partie 2 ne l'atteint pas.

```
CREATE OR REPLACE TABLE LABO09_VENTE_VERSION AS
SELECT f.SalesOrderNumber,
       f.SalesOrderLineNumber,
       f.OrderDate::date        AS jour,
       f.CustomerKey,
       f.SalesAmount,
       f.ProductKey             AS cle_portee_par_le_fait,
       ref.ProductAlternateKey  AS code_produit,
       v.ProductKey             AS cle_selon_les_dates,
       v.ListPrice              AS prix_en_vigueur,
       v.StandardCost           AS cout_en_vigueur
FROM   dbo.FactInternetSales AS f
JOIN   dbo.DimProduct AS ref ON ref.ProductKey = f.ProductKey
LEFT   JOIN LATERAL (
         SELECT p.ProductKey, p.ListPrice, p.StandardCost
```


© 2026 iAlexMG. Tous droits réservés.

```
          FROM   dbo.DimProduct AS p
          WHERE  p.ProductAlternateKey = ref.ProductAlternateKey
            AND  p.StartDate <= f.OrderDate
          ORDER  BY p.StartDate DESC
          LIMIT  1
        ) AS v ON TRUE;

SELECT COUNT(*) AS lignes,
       COUNT(cle_selon_les_dates) AS avec_version,
       COUNT(*) - COUNT(cle_selon_les_dates) AS sans_version
FROM   LABO09_VENTE_VERSION;
```

Attendu : 60 398 lignes, dont 18 410 sans version. Ces dernières sont les ventes antérieures à la première date de début enregistrée pour ce produit : la dimension prétend que le produit n'existait pas encore le jour où il a été vendu.

Le `LEFT` est ce qui vous permet de le voir. Avec une jointure interne, ces 18 410 lignes auraient disparu du résultat, et le total des ventes aurait baissé de trente pour cent sans que rien ne le signale.

3.2 La même chose en `QUALIFY`

Théorie : `Theorie_09_SQL.md`, section 6.

L'écriture ci-dessus n'est pas la seule. En voici une seconde, plus idiomatique, qui repose sur le `QUALIFY` du Cours 04. On joint sur l'inégalité, ce qui produit toutes les versions candidates, puis on ne garde que la plus récente.

```
SELECT f.SalesOrderNumber, f.SalesOrderLineNumber, f.OrderDate::date AS jour,
       v.ProductKey AS cle_selon_les_dates, v.ListPrice
FROM   dbo.FactInternetSales AS f
JOIN   dbo.DimProduct AS ref ON ref.ProductKey = f.ProductKey
LEFT   JOIN dbo.DimProduct AS v
       ON v.ProductAlternateKey = ref.ProductAlternateKey
      AND v.StartDate <= f.OrderDate
QUALIFY ROW_NUMBER() OVER (PARTITION BY f.SalesOrderNumber, f.SalesOrderLineNumber
                           ORDER BY v.StartDate DESC) = 1
LIMIT  20;
```

Les deux écritures disent la même chose. La première exprime « pour chaque vente, va chercher une ligne » ; la seconde exprime « prends toutes les candidates, garde la meilleure ». Sur un gros volume, la seconde est souvent la plus rapide sur cet entrepôt, parce qu'elle laisse le moteur travailler par blocs plutôt que ligne à ligne. Préfixez les deux de `EXPLAIN ANALYZE` et comparez le temps et les opérateurs.

Il existe une troisième écriture, la plus courte : DuckDB porte un `ASOF JOIN` natif, qui dit le geste en un mot. La dernière condition du `ON`, celle qui porte l'inégalité, est celle sur laquelle il fait la recherche du plus proche en dessous.

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT f.SalesOrderNumber, f.SalesOrderLineNumber, f.OrderDate::date AS jour,
       v.ProductKey AS cle_selon_les_dates, v.ListPrice
FROM   dbo.FactInternetSales AS f
JOIN   dbo.DimProduct AS ref ON ref.ProductKey = f.ProductKey
ASOF LEFT JOIN dbo.DimProduct AS v
       ON v.ProductAlternateKey = ref.ProductAlternateKey
      AND v.StartDate <= f.OrderDate
LIMIT  20;
```

Observez. Même résultat que les deux précédentes, sans `QUALIFY`, sans fenêtre et sans CTE. Le `LEFT` garde ici le même rôle : sans lui, les ventes sans version en vigueur disparaîtraient.

Portabilité — Ce raccourci n'est pas portable, et c'est le seul du module. Snowflake écrit `ASOF JOIN` avec une clause `MATCH_CONDITION` qui porte l'inégalité et un `ON` qui porte l'égalité de clé ; PostgreSQL n'a rien de tel et passe par le `LATERAL` du labo jumeau. En entretien, donnez d'abord la version portable, celle du `QUALIFY` ou du `LATERAL` : elle montre que vous avez compris le mécanisme, et elle fonctionne partout.

3.3 Le contrôle qui donne son sens au module

Théorie : `Theorie_09_SQL.md`, section 6.

Une question s'impose maintenant. Le fait porte déjà une `ProductKey`, donc l'alimentation a déjà résolu cet as-of join au chargement. Est-ce qu'elle a trouvé la même chose que vous ?

```sql
SELECT CASE WHEN cle_selon_les_dates IS NULL         THEN 'aucune version valide'
            WHEN cle_selon_les_dates = cle_portee_par_le_fait THEN 'accord'
            ELSE 'desaccord' END AS verdict,
       COUNT(*) AS lignes
FROM   LABO09_VENTE_VERSION
GROUP  BY 1 ORDER BY 2 DESC;
```

Attendu : 35 089 en accord, 18 410 sans version valide, 6 899 en désaccord. Autrement dit, pour une ligne de fait sur dix, la version du produit désignée par les dates de la dimension n'est pas celle que la clé technique désigne.

Ne concluez pas trop vite que l'alimentation a tort. Trois lectures se défendent, et savoir les distinguer vaut mieux que d'avoir raison tout de suite.

Lecture	Ce qu'elle suppose	Comment la départager
Les dates de la dimension sont fausses	`StartDate` serait un millésime de catalogue, pas une date de validité	comparer `StartDate` aux dates de vente réelles du même produit
L'alimentation du fait est fausse	elle aurait rattaché la version courante plutôt que la version du jour	chercher si les désaccords pointent toujours vers la version `Current`

© 2026 iAlexMG. Tous droits réservés.

Lecture	Ce qu'elle suppose	Comment la départager
Les deux décrivent des choses différentes	la clé du fait dirait « la version au moment du chargement »	le demander à celui qui a écrit l'alimentation

La deuxième requête est à votre portée :

```sql
SELECT COUNT(*) AS desaccords_vers_la_version_courante
FROM   LABO09_VENTE_VERSION AS x
JOIN   dbo.DimProduct AS p ON p.ProductKey = x.cle_portee_par_le_fait
WHERE  x.cle_selon_les_dates IS NOT NULL
  AND  x.cle_selon_les_dates <> x.cle_portee_par_le_fait
  AND  p.Status = 'Current';
```

Ce genre de contrôle est le travail quotidien d'un ingénieur analytique, et c'est exactement ce que l'anti-jointure du Cours 07 préparait. La différence est qu'ici, ce n'est pas une ligne qui manque, c'est une ligne qui est présente et fausse.

Partie 4 — Séries, axe complet et cadres

4.1 La série brute et ses trous

Théorie : `Theorie_09_SQL.md`, section 2.

```sql
CREATE OR REPLACE TABLE LABO09_VENTES_JOUR AS
SELECT f.OrderDate::date AS jour,
       SUM(f.SalesAmount) AS ventes,
       COUNT(DISTINCT f.SalesOrderNumber) AS commandes
FROM   dbo.FactInternetSales AS f
GROUP  BY 1;

SELECT COUNT(*) AS jours_actifs,
       DATEDIFF('day', MIN(jour), MAX(jour)) + 1 AS jours_de_la_plage
FROM   LABO09_VENTES_JOUR;
```

Attendu : 1 124 jours actifs pour 1 127 jours de plage. Trois jours sans aucune vente, invisibles tant qu'on ne les cherche pas.

Portabilité — L'unité de `DATEDIFF` s'écrit entre guillemets simples, `'day'`, sur DuckDB comme sur PostgreSQL, qui la traitent comme une chaîne. Snowflake accepte en plus le mot nu, `day`, d'après sa documentation, et c'est son écriture habituelle. Sans les guillemets, DuckDB cherche une colonne nommée `day` et répond `Referenced column "day" not found in FROM clause`, ce qui n'aide pas beaucoup à comprendre.

© 2026 iAlexMG. Tous droits réservés.

4.2 L'axe complet, gratuitement

Théorie : `Theorie_09_SQL.md`, section 2.

Sur l'entrepôt, l'axe existe déjà. C'est la raison d'être de `dbo.DimDate`, et c'est un avantage structurel de ce terrain sur l'autre.

```
CREATE OR REPLACE TABLE LABO09_SERIE AS
SELECT d.FullDateAlternateKey       AS jour,
       COALESCE(v.ventes, 0)        AS ventes,
       COALESCE(v.commandes, 0)     AS commandes,
       (v.jour IS NULL)             AS jour_vide
FROM   dbo.DimDate AS d
LEFT   JOIN LABO09_VENTES_JOUR AS v ON v.jour = d.FullDateAlternateKey
WHERE  d.FullDateAlternateKey BETWEEN (SELECT MIN(jour) FROM LABO09_VENTES_JOUR)
                              AND (SELECT MAX(jour) FROM LABO09_VENTES_JOUR);

SELECT COUNT(*) AS jours, COUNT_IF(jour_vide) AS jours_vides FROM LABO09_SERIE;
```

Attendu : 1 127 jours, dont 3 vides. Le `BETWEEN` est légitime ici, contrairement à celui de la partie 2 : il porte sur une colonne de type `DATE` dont chaque valeur est un jour entier, pas sur un intervalle de validité.

4.3 La moyenne mobile

Théorie : `Theorie_09_SQL.md`, section 4.

```
SELECT jour, ventes,
       AVG(ventes) OVER (ORDER BY jour ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS mm7
FROM   LABO09_SERIE
ORDER  BY jour
LIMIT  40;
```

`ROWS` suffit et donne le bon résultat, parce que l'axe est complet : une ligne égale un jour. C'est la parade de la section 4 de la théorie, appliquée. Le labo jumeau, lui, écrit le cadre en durée sur PostgreSQL et compare les deux sur une série trouée.

Portabilité — Le cadre glissant exprimé en durée, `RANGE BETWEEN INTERVAL 6 DAY PRECEDING`, passe ici comme sur PostgreSQL : essayez-le à la place du `ROWS` ci-dessus, vous obtiendrez la même colonne. Il est plus restreint sur d'autres moteurs, et son support y a bougé au fil des versions. Quoi qu'il en soit, l'écriture en `ROWS` reste la bonne réponse sur un axe complet : les deux cadres y décrivent le même ensemble, et celui en lignes voyage mieux.

4.4 Rééchantillonner, et les quatre mesures d'une période

Théorie : `Theorie_09_SQL.md`, section 5.

```
WITH par_mois AS (
    SELECT jour, ventes,
```


© 2026 iAlexMG. Tous droits réservés.

```
            DATE_TRUNC('month', jour) AS mois,
            FIRST_VALUE(ventes) OVER (PARTITION BY DATE_TRUNC('month', jour) ORDER BY jour
                                      ROWS BETWEEN UNBOUNDED PRECEDING
                                               AND UNBOUNDED FOLLOWING) AS ouverture,
            LAST_VALUE(ventes)  OVER (PARTITION BY DATE_TRUNC('month', jour) ORDER BY jour
                                      ROWS BETWEEN UNBOUNDED PRECEDING
                                               AND UNBOUNDED FOLLOWING) AS cloture
        FROM   LABO09_SERIE
    )
    SELECT mois,
           SUM(ventes)          AS total,
           MIN(ventes)          AS plus_bas,
           MAX(ventes)          AS plus_haut,
           MIN_BY(jour, ventes) AS jour_du_creux,
           MAX_BY(jour, ventes) AS jour_du_pic,
           MIN(ouverture)       AS ouverture,
           MIN(cloture)         AS cloture
    FROM   par_mois
    GROUP  BY mois
    ORDER  BY mois;
```

Deux points de forme. La fenêtre se calcule après le `GROUP BY`, donc l'ouverture et la clôture ne peuvent pas se demander dans la même requête que les agrégats : il faut la couche de CTE, ce qui est l'ordre d'exécution du Cours 01 qui refait surface. Et comme les deux valeurs sont constantes à l'intérieur d'un mois, `MIN` sur elles ne fait que les faire traverser l'agrégation.

`MIN_BY` et `MAX_BY` répondent littéralement à « la valeur de x pour la ligne où y est extrême », ce qu'aucun agrégat ordinaire ne sait dire. Retenez surtout le cadre explicite dans la fonction de fenêtrage : sans lui, le piège du Cours 04 se réveille et la clôture vaut la ligne courante.

Passez à une période que `DATE_TRUNC` ne connaît pas, la quinzaine, sur les deux premiers mois de 2013 :

```
SELECT TIME_BUCKET(INTERVAL '14 days', OrderDate) AS quinzaine,
       COUNT(*)                                   AS lignes,
       ROUND(SUM(SalesAmount), 2)                 AS ventes
FROM   FactInternetSales
WHERE  OrderDate >= '2013-01-01' AND OrderDate < '2013-03-01'
GROUP  BY ALL
ORDER  BY quinzaine;
```

Attendu : cinq quinzaines, qui commencent toutes un lundi, du 31 décembre 2012 au 25 février 2013, avec 581, 631, 1 703, 1 703 et 497 lignes. La première et la dernière débordent de la plage filtrée : une période de largeur libre ne s'aligne pas sur le mois. Donnez maintenant l'origine qu'aurait prise une écriture tirée d'ailleurs, le 1er janvier 2000 :

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT TIME_BUCKET(INTERVAL '14 days', OrderDate, TIMESTAMP '2000-01-01') AS quinzaine,
       COUNT(*)                                                           AS lignes
FROM   FactInternetSales
WHERE  OrderDate >= '2013-01-01' AND OrderDate < '2013-03-01'
GROUP  BY ALL
ORDER  BY quinzaine;
```

Attendu : cinq quinzaines aussi, mais qui commencent un samedi, le 29 décembre, et qui comptent 499, 633, 1 522, 1 716 et 745 lignes. Le même total, d'autres bornes, d'autres chiffres par période, et pas une erreur : c'est tout le piège de l'origine. `TIME_BUCKET` sans origine prend le lundi 3 janvier 2000 ; `DATE_BIN`, au labo jumeau, n'en a pas par défaut et vous oblige à la donner.

4.5 Variation et rendement

Théorie : `Theorie_09_SQL.md`, section 3. Théorie : `Theorie_02_SQL.md`, section 4.

```sql
WITH mensuel AS (
    SELECT DATE_TRUNC('month', jour) AS mois, SUM(ventes) AS ventes
    FROM   LABO09_SERIE GROUP BY 1
)
SELECT mois, ventes,
       ventes - LAG(ventes) OVER (ORDER BY mois) AS variation,
       ROUND(100 * (ventes - LAG(ventes) OVER (ORDER BY mois))
             / NULLIF(LAG(ventes) OVER (ORDER BY mois), 0), 1) AS variation_pct,
       LN(ventes / NULLIF(LAG(ventes) OVER (ORDER BY mois), 0)) AS rendement_log
FROM   mensuel ORDER BY mois;
```

Regardez le premier et le dernier mois : ils sont tronqués, et leur variation n'a aucun sens économique. C'est le rappel de la partie 1, et c'est la première chose qu'un lecteur pressé interprétera de travers.

Vérifiez ensuite la propriété qui justifie le rendement logarithmique :

```sql
WITH mensuel AS (
    SELECT DATE_TRUNC('month', jour) AS mois, SUM(ventes) AS ventes
    FROM   LABO09_SERIE GROUP BY 1
),
rend AS (
    SELECT mois, ventes, LN(ventes / NULLIF(LAG(ventes) OVER (ORDER BY mois), 0)) AS r
    FROM   mensuel
)
SELECT SUM(r)                                        AS somme_des_rendements_log,
       LN(MAX_BY(ventes, mois) / MIN_BY(ventes, mois)) AS rendement_total
FROM   rend;
```


© 2026 iAlexMG. Tous droits réservés.

Les deux nombres coïncident. C'est cela, s'additionner, et aucun rendement simple ne le ferait.

Partie 5 — Quantiles, sessions et cohortes

5.1 Médiane et déciles

Théorie : `Theorie_09_SQL.md`, section 8.

```sql
SELECT MEDIAN(SalesAmount)                                          AS mediane_raccourci,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY SalesAmount)     AS mediane_interpolee,
       PERCENTILE_DISC(0.5) WITHIN GROUP (ORDER BY SalesAmount)     AS mediane_observee,
       PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY SalesAmount)     AS decile_9,
       AVG(SalesAmount)                                             AS moyenne
FROM   dbo.FactInternetSales;
```

Deux choses à voir. `MEDIAN` et `PERCENTILE_CONT(0.5)` donnent le même nombre, ce qui confirme que le premier est un raccourci. Et la moyenne est très supérieure à la médiane : la distribution est tirée vers le haut par les vélos, alors que la masse des lignes est faite d'accessoires. Publier la moyenne seule sur ce jeu de données décrit mal ce qui s'y passe.

```sql
-- La version discrète renvoie une valeur qui existe. Vérifiez-le.
SELECT COUNT(*) AS lignes_a_ce_montant
FROM   dbo.FactInternetSales
WHERE  SalesAmount = (SELECT PERCENTILE_DISC(0.5) WITHIN GROUP (ORDER BY SalesAmount)
                      FROM dbo.FactInternetSales);
```

5.2 Les sessions d'achat d'un client

Théorie : `Theorie_09_SQL.md`, section 7.

```sql
WITH commandes AS (
    SELECT DISTINCT CustomerKey, SalesOrderNumber, OrderDate::date AS jour
    FROM   dbo.FactInternetSales
),
marque AS (
    SELECT CustomerKey, SalesOrderNumber, jour,
           CASE WHEN DATEDIFF('day', LAG(jour) OVER (PARTITION BY CustomerKey ORDER BY jour),
                                jour) > 120
                THEN 1 ELSE 0 END AS rupture
    FROM   commandes
)
SELECT CustomerKey, jour, SalesOrderNumber,
```


© 2026 iAlexMG. Tous droits réservés.

```
        SUM(rupture) OVER (PARTITION BY CustomerKey ORDER BY jour
                           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS session
FROM    marque
WHERE   CustomerKey = 11000
ORDER   BY jour;
```

Le `DISTINCT` de la première CTE est nécessaire : le fait porte une ligne par article, et sans lui une commande de trois articles compterait pour trois événements. C'est le grain qu'on redresse avant de raisonner, pas après.

5.3 La cohorte par mois de première commande

Théorie : `Theorie_09_SQL.md`, section 9.

```
WITH premiere AS (
    SELECT CustomerKey, DATE_TRUNC('month', MIN(OrderDate))::date AS mois_cohorte
    FROM   dbo.FactInternetSales GROUP BY CustomerKey
),
activite AS (
    SELECT f.CustomerKey, p.mois_cohorte,
           DATE_TRUNC('month', f.OrderDate)::date AS mois_actif
    FROM   dbo.FactInternetSales AS f
    JOIN   premiere AS p ON p.CustomerKey = f.CustomerKey
),
grille AS (
    SELECT mois_cohorte,
           DATEDIFF('month', mois_cohorte, mois_actif) AS rang,
           COUNT(DISTINCT CustomerKey)                 AS clients
    FROM   activite GROUP BY 1, 2
)
SELECT mois_cohorte, rang, clients,
       ROUND(100.0 * clients / MAX(CASE WHEN rang = 0 THEN clients END)
                               OVER (PARTITION BY mois_cohorte), 1) AS retention_pct
FROM   grille
WHERE  rang <= 6
ORDER  BY mois_cohorte, rang;
```

Comparez ensuite `mois_cohorte` avec la colonne `DateFirstPurchase` de `dbo.DimCustomer`, qui prétend dire la même chose :

```
WITH premiere AS (
    SELECT CustomerKey, MIN(OrderDate)::date AS calculee
    FROM   dbo.FactInternetSales GROUP BY CustomerKey
)
SELECT COUNT(*) AS clients,
```


© 2026 iAlexMG. Tous droits réservés.

```
        COUNT_IF(c.DateFirstPurchase <> p.calculee) AS ecarts
FROM    premiere AS p
JOIN    dbo.DimCustomer AS c ON c.CustomerKey = p.CustomerKey;
```

Une colonne pré-calculée dans une dimension est un raccourci commode, et c'est aussi une valeur qui peut se désynchroniser du fait. Vous savez maintenant vérifier avant de vous en servir.

Deux réserves, les mêmes que sur l'autre terrain. Les rangs sans client actif n'ont pas de ligne, et la courbe saute le point. Et les dernières cohortes n'ont pas eu six mois pour revenir, donc leur rétention au rang 6 est inconnue, pas nulle.

Partie 6 — Ce que l'entrepôt vous a montré

Théorie : `Theorie_09_SQL.md`, section 10.

Vous verrez la colonne PostgreSQL se remplir en faisant `Labo_09_OLTP_SQL.md`. Notez déjà ce que ce terrain vous a appris.

Observation	**Sur l'entrepôt**
L'axe des dates	déjà là, complet, dans `dbo.DimDate` ; rien à générer
Le grain du fait	une ligne par article, à redresser avant de compter des commandes
Où vit l'historique	replié dans `dbo.DimProduct`, une ligne par version, clé métier `ProductAlternateKey`
La qualité des intervalles	200 dates de fin sur 200 sont antérieures à leur début
L'écriture par intervalle fermé	renvoie zéro ligne, sans erreur
L'as-of join par la date de début	fonctionne, et rend visibles 18 410 ventes sans version valide
Le contrôle qui compte	l'as-of join recalculé contredit la clé du fait sur 6 899 lignes
Le cadre glissant en durée	accepté, comme sur PostgreSQL ; on complète l'axe et on écrit `ROWS` quand même, pour que le code voyage
Les raccourcis du moteur	`MEDIAN`, `MIN_BY`, `MAX_BY`, `COUNT_IF`, `DATEDIFF`
La journée d'un changement d'heure	25 heures générées, mais une soustraction qui rend `1 day` ; `DATE_DIFF('hour', …)` dit 25

© 2026 iAlexMG. Tous droits réservés.

Observation	Sur l'entrepôt
Une période de largeur libre	`TIME_BUCKET`, dont l'origine par défaut est le lundi 3 janvier 2000

Nettoyage

```
DROP TABLE IF EXISTS LABO09_VENTE_VERSION;
DROP TABLE IF EXISTS LABO09_VENTES_JOUR;
DROP TABLE IF EXISTS LABO09_SERIE;
```

Et ensuite ?

Vous savez rattacher à un événement la valeur qui était vraie au moment où il s'est produit, et vous avez vu pourquoi l'écriture qui s'appuie sur la seule date de début est la seule à survivre à une donnée imparfaite.

Faites maintenant `Labo_09_OLTP_SQL.md`, qui pose les mêmes questions sur un historique propre. Le contraste est le sujet du module autant que l'as-of join lui-même : la table dédiée du transactionnel et la dimension repliée de l'entrepôt ne se cassent pas de la même façon.

Prochaine étape : les exercices, dans `Exercices_corriges_09_SQL.md`.

© 2026 iAlexMG. Tous droits réservés.